

# Deadlock-Free Adaptive Routing and Bubble Flow Control for a 3D Torus in Garnet

Lab 4 Project Report

August 27, 2026

## Abstract

This project adds parameterized 3D Mesh and 3D Torus topologies to gem5 Garnet and implements three routing modes: Mesh3D XYZ, Torus3D minimal dimension-order routing (DOR), and a minimal adaptive Torus mode with a one-way Mesh escape virtual-channel (VC) class. The default design reserves one of four VCs as an escape path and keeps the other three adaptive. On the 4x4x4 transpose workload, it sustains 0.498859 packets/node/Ruby-cycle, versus 0.237455 for Torus DOR and 0.209570 for Mesh3D XYZ. A fixed-budget VC ablation shows that one escape VC retains the performance of the unsafe no-escape upper bound, while allocating more VCs to escape unnecessarily reduces throughput. On the same substrate, the project adds optional Critical Bubble Scheme (CBS) flow control, which makes the otherwise unprotected Torus DOR mode constructively deadlock-free without reserving any VC and raises its maximum stable transpose throughput by about 15% (from 0.224067 to 0.257509 under that sweep’s coarser rate grid), with negligible performance cost on the other patterns (at most a 0.049-cycle full-load latency difference). The report separates topology benefit from routing benefit and gives the channel-dependency argument for adaptive progress and the buffer-occupancy argument for CBS.

## 1 Goal and Scope

The goal is not to present the 3D Torus as a new topology. Instead, this work implements and evaluates, on one Garnet substrate, two layers that address the cyclic dependencies that Torus wrap channels can introduce. The routing layer implements congestion-aware minimal adaptive routing over a disjoint, connected 3D Mesh escape subnetwork; once a packet selects that class, it follows non-wrap XYZ routing and never returns to an adaptive VC. The flow-control layer instead protects the deterministic Torus DOR mode with the Critical Bubble Scheme (CBS) [?]: no VC is reserved, and safety comes from keeping at least one free buffer slot on every directed ring. The two layers are gated independently, share the topology, metrics, and sweep structure, and are compared against each other in the flow-control sections.

The evaluation uses 64 routers arranged as 4x4x4. Each Garnet physical neighbour pair is represented by two directed internal links. The 3D Mesh has  $3(n-1)n^2 = 144$  undirected neighbour pairs at  $n = 4$ ; the 3D Torus has  $3n^3 = 192$ , a 33.3% increase. Therefore Mesh-versus-Torus results include wrap-link cost and topology effects. Torus DOR versus Torus adaptive holds the topology, link count, clocks, packet type, and total VC count fixed, so it isolates the routing-policy effect.

## 2 Implementation

Router IDs use  $zXY+yX+x$ . The project adds `Torus3D.py` and `Mesh3D.py`, command-line dimensions `-torus-x/y/z`, and these routing-algorithm IDs:

| ID | Topology | Routing policy                                |
|----|----------|-----------------------------------------------|
| 3  | Torus3D  | Deterministic minimal XYZ DOR                 |
| 4  | Torus3D  | Minimal adaptive routing with Mesh escape VCs |
| 5  | Mesh3D   | Deterministic non-wrap XYZ                    |

ID 3 can additionally run under CBS flow control, gated by `-enable-cbs` and described in the flow-control sections below; with the gate closed, every mode behaves exactly as described here.

Let  $V$  be the number of VCs per vnet and  $E$  the number of escape VCs. The adaptive range is  $[0, V - E)$  and the escape range is  $[V - E, V)$ . The implementation validates  $0 \leq E < V$  for adaptive routing; safety requires  $E \geq 1$ . New packets enter the adaptive range. At each adaptive hop, all minimal Torus directions are considered, then candidates with the most usable downstream adaptive VCs are preferred, with a pseudo-random tie break. If no minimal adaptive output VC is usable, a head flit may enter an escape VC.

The escape route intentionally ignores Torus wrap links and advances signed coordinates in X, then Y, then Z. This makes its path a route in a connected 3D Mesh. The switch allocator enforces the crucial one-way transition:

$$A \rightarrow A, \quad A \rightarrow E, \quad E \rightarrow E, \quad \text{but never} \quad E \rightarrow A.$$

### VC-class routing state machine

Blue: minimal congestion-aware path. Green: connected progress path.

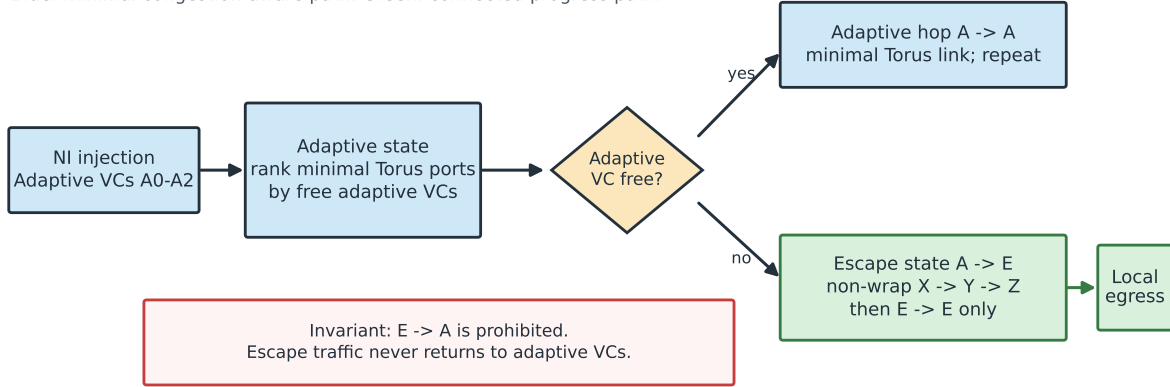


Figure 1: Routing and VC-class state machine for the default 3A/1E split. Blue is the performance path; green is the connected acyclic progress path. The red transition is absent from the implementation.

## 3 Progress Argument

The argument follows the connected acyclic escape-subset condition for adaptive wormhole routing due to Duato [?]. It is a proof about the route and VC-allocation relation, not an inference from a finite simulation.

**Definition 1** (Escape channel set). *Let  $C_E$  be all directed physical links paired with the Escape VC range. Its channel-dependency graph (CDG) has one vertex per such channel and an edge  $c \rightarrow d$  if one legal Escape route can hold  $c$  while requesting  $d$ .*

**Lemma 1** (Connectivity). *For every router pair  $(s, t)$ , a route using only  $C_E$  exists.*

*Proof.* The Escape policy changes the signed X coordinate until it equals the destination X coordinate, then Y, then Z. Each step stays within the finite non-wrap 3D Mesh, which is connected. The local egress terminates the route.  $\square$

**Lemma 2** (Acyclicity). *The CDG induced by  $C_E$  is acyclic.*

*Proof.* Within one dimension, an Escape route has a fixed sign and strictly advances toward its destination; it cannot reverse or use a wrap link. It therefore cannot make a same-dimension cycle. A dimension change is only X to Y or Y to Z, never backwards. Order a dependency first by dimension (X, Y, Z), then by coordinate progress in its fixed sign. Every legal CDG edge strictly advances this lexicographic order, so a directed cycle is impossible.  $\square$

## Escape-channel dependency graph: a strict ordering

Every legal dependency advances dimension or monotone coordinate rank.

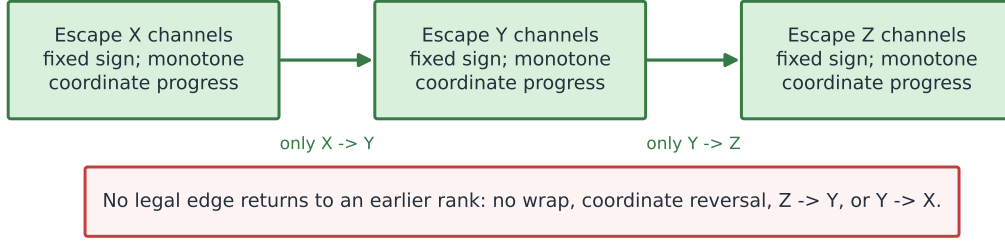


Figure 2: Projection of the Escape-channel dependency graph. The omitted backward and wrap dependencies are exactly what makes the subset acyclic.

**Theorem 1** (Deadlock freedom for  $E \geq 1$ ). *With a nonempty Escape range, the implemented route and VC-allocation relation is deadlock-free, assuming finite buffers, reliable credits, and a fair switch arbiter.*

*Proof.* Assume a protocol deadlock exists. A blocked Escape header can request only its next Escape channel: the allocator never assigns an Adaptive VC to an Escape input. Following its wait dependencies can therefore never leave  $C_E$ ; in a finite deadlock it would contain a cycle in  $C_E$ , contradicting the preceding lemma.

Now consider a blocked Adaptive header. If any minimal Adaptive output VC is free, it can advance and is not deadlocked. Otherwise, the allocator tests its unique non-wrap Escape next hop. If that Escape VC is free, it advances into  $C_E$ . If it is occupied, its wait dependency enters  $C_E$ . Thus every otherwise closed Adaptive wait set has an outgoing dependency into  $C_E$ . Because the implementation contains no  $C_E \rightarrow C_A$  assignment, a finite dependency chain that enters  $C_E$  cannot form an all-Adaptive cycle and cannot form an Escape cycle by the acyclicity lemma. This contradicts the assumed deadlock.  $\square$

The experiment with  $E = 0$  is deliberately labelled *unsafe*. Completion of a finite run without an escape class is not a proof of deadlock freedom.

## 4 Methodology

All main points use four VCs per vnet, single-flit vnet-0 control traffic, and 10,000 Ruby cycles. Ruby runs at 2 GHz and the system clock at 1 GHz. For injection rate  $r$  in packets/node/system-cycle, expected packets are  $N \cdot (\text{simCycles}/2) \cdot r$ . Throughput is received packets per node per Ruby cycle; source acceptance is injected divided by expected packets. A point is classified as stalled when source acceptance is below 0.9.

The 600-point sweep covers uniform random, 3D neighbour, 3D tornado, and 3D transpose traffic. A separate X-opposite pattern sends  $(x, y, z)$  to  $(x + X/2 \bmod X, y, z)$  for even  $X$ , stressing equal-cost wrap-direction ties. It is a useful validation traffic but does not replace the dependency argument.

## 5 Results

At low offered load, Torus routes reduce hop count relative to the Mesh. The more important result occurs for transpose traffic, where adaptive routing avoids deterministic route concentration. Table ?? reports the measured saturation summary.

Table 1: Transpose traffic, 4x4x4, four VCs per vnet.

| Mode                    | Low-load latency | Hops  | Max stable throughput | First stalled rate |
|-------------------------|------------------|-------|-----------------------|--------------------|
| Mesh3D XYZ              | 12.562           | 3.763 | 0.209570              | 0.48               |
| Torus3D DOR             | 11.031           | 3.006 | 0.237455              | 0.50               |
| Torus3D adaptive/escape | 11.026           | 3.006 | 0.498859              | not observed       |

The adaptive design reaches 2.38x Mesh throughput and 2.10x Torus DOR throughput. The DOR comparison shows that the main high-load gain is not merely the Torus wrap links: both Torus modes have the same links, yet only the adaptive policy sustains the generator's maximum offered load.

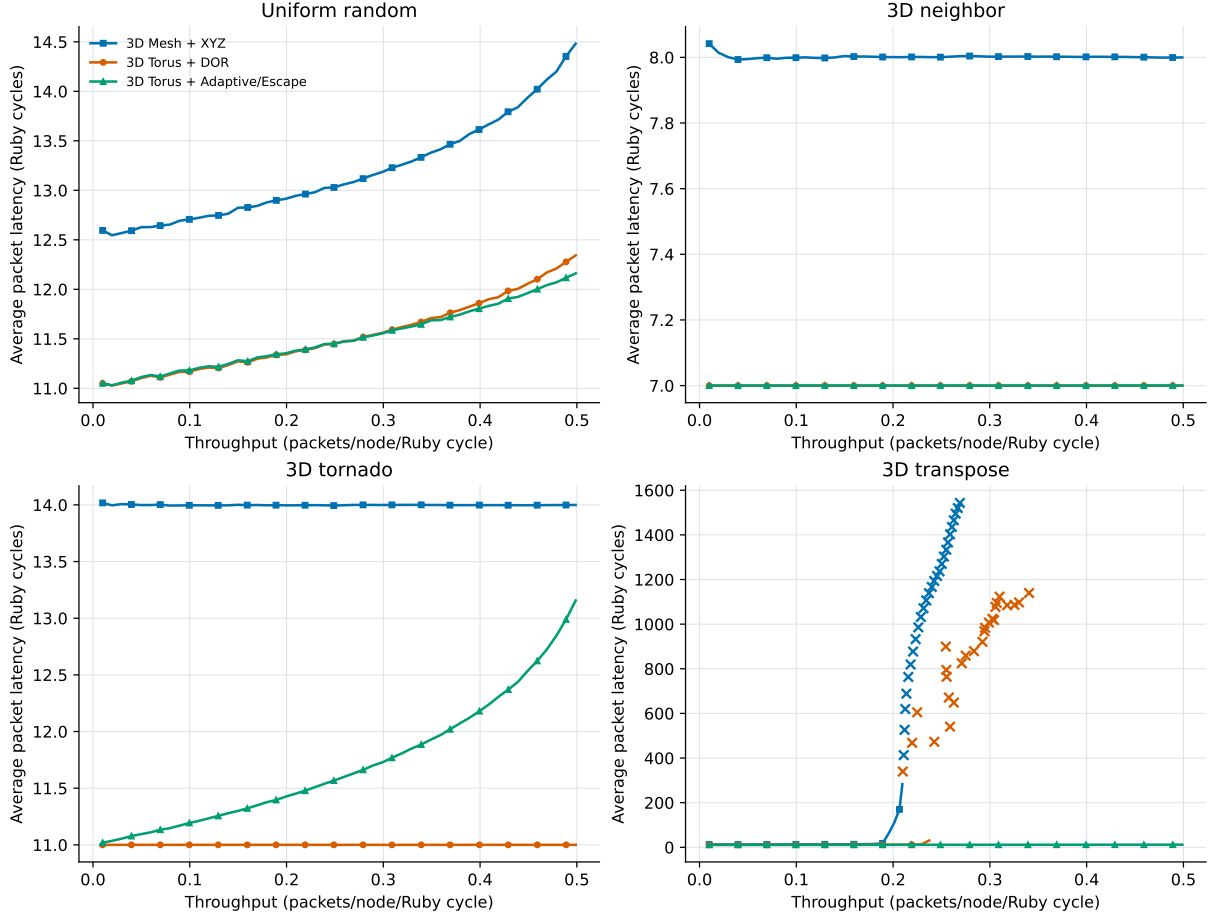


Figure 3: Latency and delivered throughput for the four main traffic patterns.

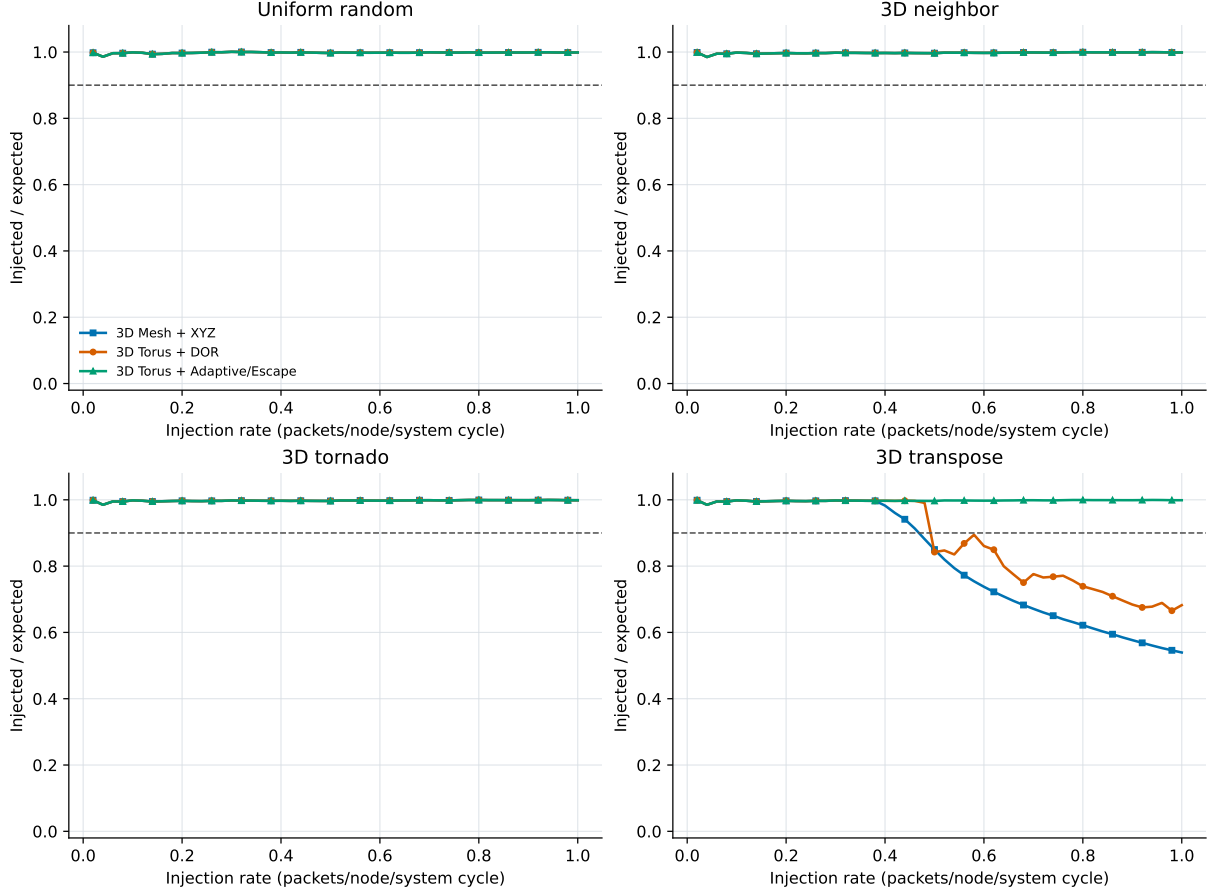


Figure 4: Source acceptance distinguishes offered traffic from traffic that the NI can inject.

Escape use remains low at full offered load: 0.34% of classified hops for transpose, 0.52% for uniform random, and 4.80% for tornado. It is therefore a rarely used progress path rather than the normal data path.

## 6 Fixed-VC Ablation

To test whether the default partition wastes resources, total VCs remain fixed at four while the split varies. The no-escape result is a performance-only upper bound; it intentionally carries no deadlock-freedom guarantee.

Table 2: Transpose at injection rate 1.0, fixed total of four VCs.

| Adaptive/escape | Acceptance | Throughput | Latency  | Safety guarantee |
|-----------------|------------|------------|----------|------------------|
| 4A/0E           | 0.999      | 0.498866   | 11.782   | no               |
| 3A/1E           | 0.999      | 0.498859   | 11.836   | yes              |
| 2A/2E           | 0.773      | 0.386294   | 1138.807 | yes              |
| 1A/3E           | 0.396      | 0.197884   | 3023.314 | yes              |

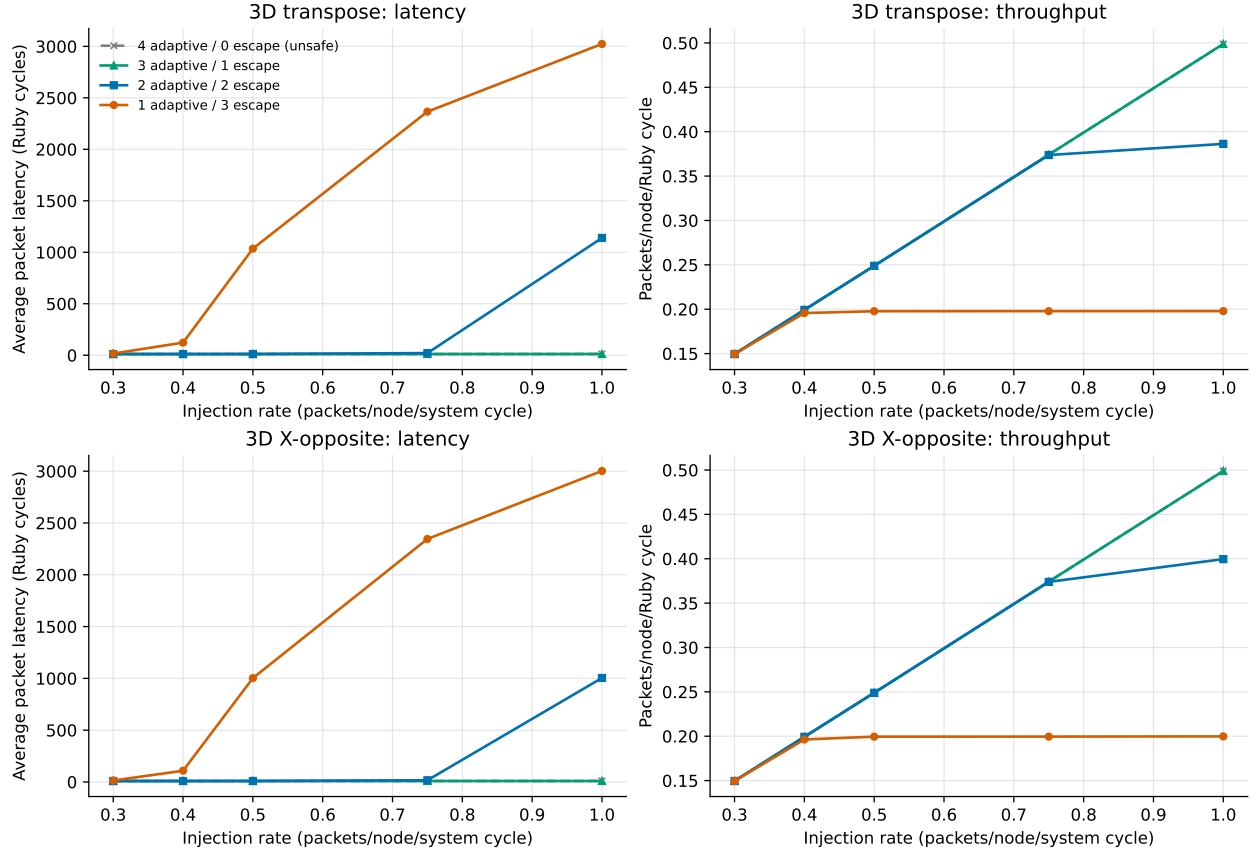


Figure 5: Fixed-total-VC ablation under transpose and X-opposite traffic.

One escape VC is sufficient for the acyclic fallback and preserves the no-escape throughput envelope. Reserving more escape VCs starves the adaptive class, causing much earlier queueing and saturation. The X-opposite result shows the same tradeoff, which makes 3A/1E the appropriate default.

## 7 Flow Control: Critical Bubble Scheme

The escape design above buys safety by partitioning channels: one of four VCs per vnet is reserved and, as measured, rarely used. The flow-control layer takes the complementary route for the unprotected Torus DOR mode (routing ID 3), whose completion of a finite run is otherwise no proof of deadlock freedom. The Critical Bubble Scheme (CBS) [?] keeps all four VCs usable and instead maintains a buffer-occupancy invariant: every directed ring always retains at least one free buffer slot (a bubble). A ring that can never fill cannot close a circular wait. Classic bubble flow control imposes this locally, demanding two free downstream slots at every ring entry; CBS explicitly marks the one slot that must survive (the critical bubble) and lets the mark travel around the ring like a token, so only entries adjacent to the mark pay the two-slot condition and in-ring traffic is unrestricted.

The implementation targets the single-flit control vnets (`buffers_per_ctrl_vc=1`, so one slot is one VC). At initialization, every directed ring (six directions per vnet) receives one mark at the facing input port of its coordinate-0 router. The registry is a single source of truth in `GarnetNetwork`, `m_cbs_mark[router][dirn][vnet]`, with direction indices E, W, N, S, U, D numbered 0–5. The switch allocator enforces two rules, keyed on whether the downstream input port of the requested output holds the mark:

| Rule  | Transfer                       | Effect                                         |
|-------|--------------------------------|------------------------------------------------|
| Entry | injection or dimension change  | marked port admits only with $\geq 2$ free VCs |
| Shift | straight through within a ring | filling the last slot moves the mark upstream  |

The entry rule runs in the non-adaptive branch of `send_allowed`: a transfer that enters a ring while the downstream input port holds the mark is granted only if that port still exposes at least two free VCs for the

vnet; otherwise the request loses the cycle, counted as `cbs_entry_blocks`. An entry therefore never consumes the marked last free slot. The shift rule runs after `decrement_credit`: when a straight-through in-ring transfer fills the last downstream slot, the mark moves to the slot the packet has just vacated, counted as `cbs_mark_moves`. In-ring traffic may eat the last slot, but the number of free slots on the ring is conserved; only their position changes.

All changes are gated by `-enable-cbs`, default off; with the gate closed, every mode, including routing ID 4, is bit-identical to the pre-CBS implementation. The change set is `GarnetNetwork.py` and `Network.py` for the option, `GarnetNetwork.hh/.cc` for the mark registry, helper functions, statistics, and `fatal_if` configuration checks, `OutputUnit.hh/.cc` for `count_free_vcs`, and `SwitchAllocator.hh/.cc` for the two rules. CBS requires routing ID 3, nonzero `-torus-x/y/z`, and at least two VCs per vnet (an entry point facing the mark could otherwise never see two free slots), and it rejects `-wormhole`; any violation is a fatal configuration error rather than silent misbehaviour.

## 8 CBS Correctness Argument

As with the escape argument, this is a claim about the allocation rules, not an inference from a finite simulation. Slots and VCs coincide on the control vnets, so the invariant is stated per directed ring and per vnet.

**Definition 2** (Critical bubble invariant). *For every directed ring and control vnet, the marked input port on that ring holds at least one free VC.*

**Lemma 3** (Preservation). *Every switch-allocator grant preserves the critical bubble invariant.*

*Proof.* A grant consumes one VC at its downstream input port. If that port is unmarked, the marked port is untouched. If it is marked and the transfer is a ring entry, the entry rule required two free VCs, so one survives. If it is marked and the transfer is in-ring, the packet’s departure frees exactly one VC at its previous input port on the same ring; when the grant fills the last marked slot, the shift rule moves the mark onto that freed slot, so the newly marked port holds a free VC. The rules test the local `OutputUnit` credit view, in which allocations are visible immediately and releases are delayed by credit return; this view can only underestimate downstream free VCs, so the check errs strictly toward blocking and no credit race weakens the invariant. Finally, one link delivers at most one packet per cycle to a given input port, so the mark’s read-modify-write is unraced.  $\square$

**Lemma 4** (Ring progress). *Under the invariant, no directed ring can sustain a circular wait among single-flit DOR packets.*

*Proof.* The invariant keeps at least one slot on the ring free. The packet immediately upstream of a free slot whose next hop is in-ring requests exactly that slot and is eventually granted by a fair arbiter; its advance moves the free slot one hop upstream. A packet whose next hop leaves the ring (a dimension change or ejection) does not wait on the ring at all. Circulating the free slot therefore eventually offers every ring packet its in-ring resource, so a wait cycle closed entirely on the ring is impossible.  $\square$

**Theorem 2** (Deadlock freedom for DOR under CBS). *With CBS enabled, single-flit DOR control traffic is deadlock-free, assuming finite buffers, reliable credits, a fair switch arbiter, and consuming ejection.*

*Proof.* DOR corrects X, then Y, then Z, and never reverses direction inside a dimension, so every wait dependency either stays on one directed ring or advances in the dimension order; no dependency returns to an earlier dimension. A cyclic wait would therefore have to close inside a single directed ring, which the ring-progress lemma excludes. Inducting downward from Z, packets in the last dimension either eject or advance on a never-full ring; a packet blocked at a dimension change waits on a higher-dimension ring whose own progress eventually satisfies the entry rule. Hence no deadlock exists.  $\square$

Bare DOR with the gate closed remains labelled unsafe. Its finite runs, including the 200,000-cycle probe reported below, are evidence about those runs, not a guarantee; CBS converts that probabilistic observation into a constructive one.

## 9 CBS Evaluation

The CBS sweep reuses the 4x4x4 substrate, clocks, metrics, and stall definition of the methodology above. Three modes run over the four main patterns at twenty injection rates from 0.05 to 1.00, or 240 points: unprotected

Torus DOR (`torus3d_dor`), the same routing under CBS (`torus3d_dor_cbs`), and the adaptive/escape design (`torus3d_adaptive_escape`). Table ?? reports the saturation summary for transpose, the only pattern where the three modes separate in saturation behaviour (on tornado they separate only in loaded latency, as noted below). The bare-DOR figures differ slightly from Table ?? because this sweep uses a coarser 0.05 rate grid, so its last stable DOR point sits at rate 0.45 rather than 0.48.

Table 3: Transpose traffic under the CBS sweep, 4x4x4, four VCs per vnet.

| Mode                    | Low-load latency | Max stable throughput | First stalled rate |
|-------------------------|------------------|-----------------------|--------------------|
| Torus3D DOR             | 11.062           | 0.224067              | 0.50               |
| Torus3D DOR + CBS       | 11.062           | 0.257509              | 0.60               |
| Torus3D adaptive/escape | 11.042           | 0.498859              | not observed       |

CBS raises DOR’s maximum stable transpose throughput from 0.224067 to 0.257509, a 14.9% increase, and moves the stall point from rate 0.50 to 0.60. The effect at the old stall point is large: at rate 0.5, average packet latency falls from 339.679 to 98.080 cycles and source acceptance rises from 0.842 to 0.972. The mechanism is that the entry rule defers injection and dimension changes exactly when a ring is about to fill, which acts as admission control ahead of saturation and suppresses the growth of saturation trees, consistent with the bubble flow-control literature. On the other three patterns CBS is free in performance terms: neighbour and tornado record zero CBS events throughout; uniform random records nonzero counters (peaks of 16,176 entry blocks and 6,059 mark moves) yet identical stable throughput and a full-load latency difference of 0.049 cycles. Tornado also shows what CBS preserves: adaptive routing’s latency rises to 13.156 at full load while DOR and DOR+CBS hold 11.0.

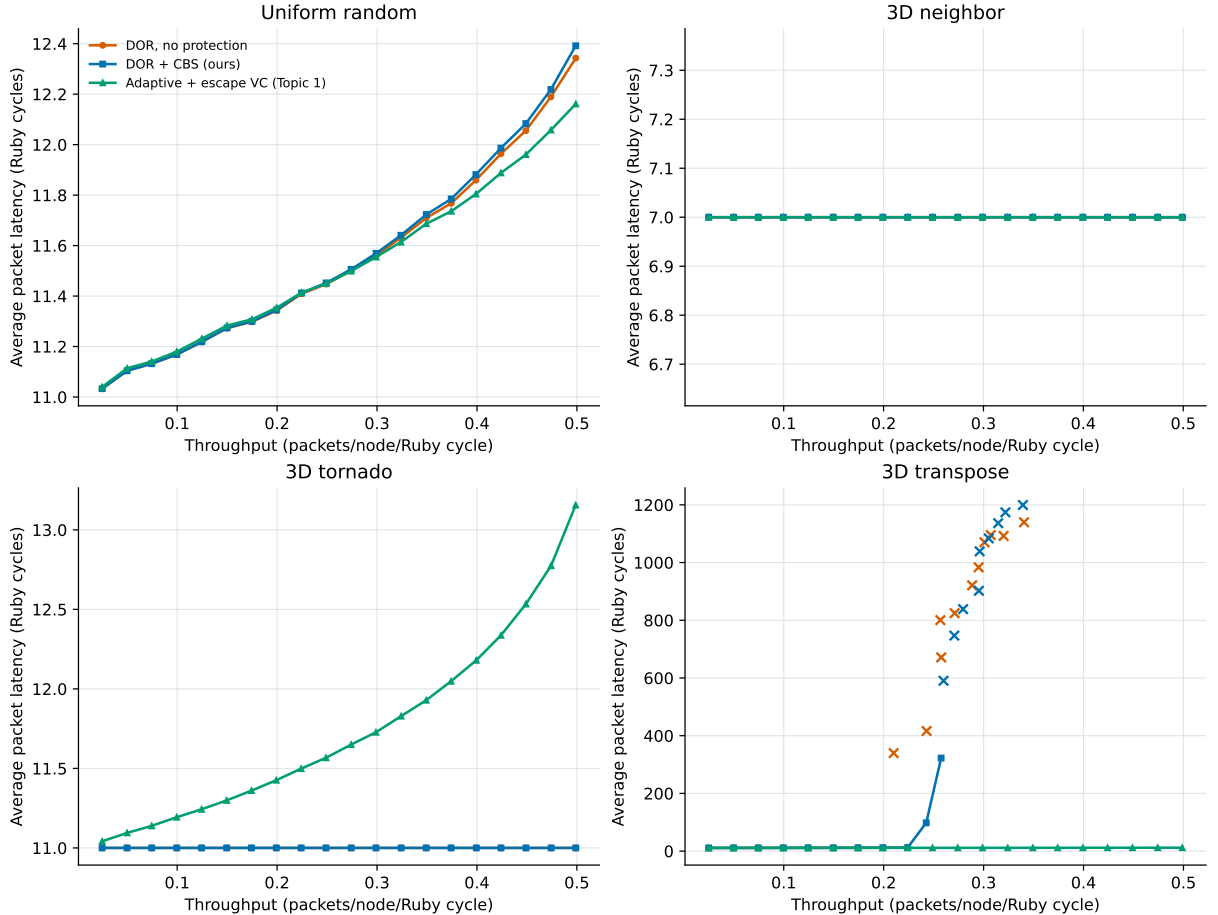


Figure 6: Latency versus offered load for the three modes over the four main traffic patterns. The CBS curve leaves bare DOR only on transpose, where it delays saturation.



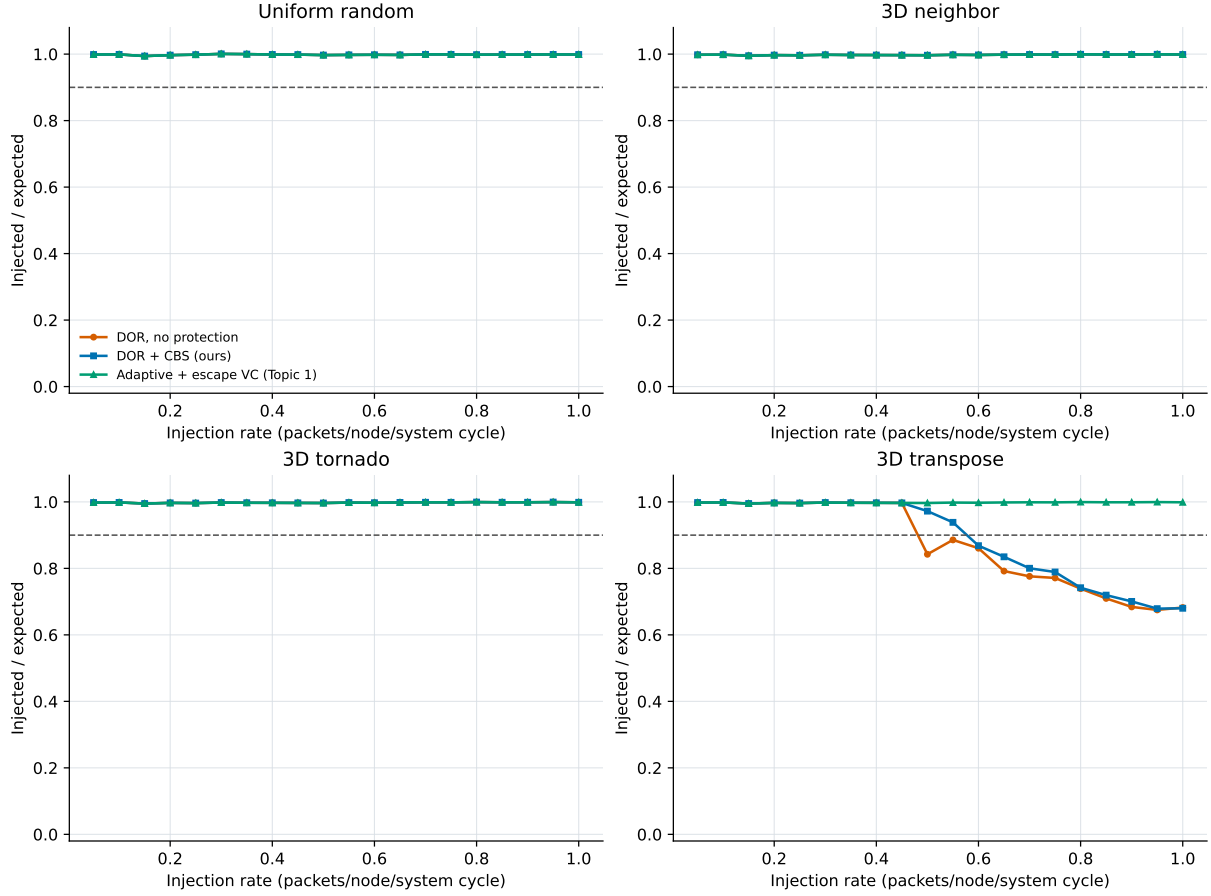


Figure 7: Source acceptance under the CBS sweep. CBS moves the transpose stall point from rate 0.5 to 0.6 and matches bare DOR elsewhere.

The CBS counters make the mechanism observable rather than inferred. `cbs_entry_blocks` grows steeply through transpose saturation and peaks at 282,418 at rate 0.95. `cbs_mark_moves` peaks at 4,380 at the saturation knee, rate 0.5, and then falls back (1,015 at 0.55, 974 at 0.6): once rings stay persistently full, the fill-the-last-slot event that moves the mark becomes rare.

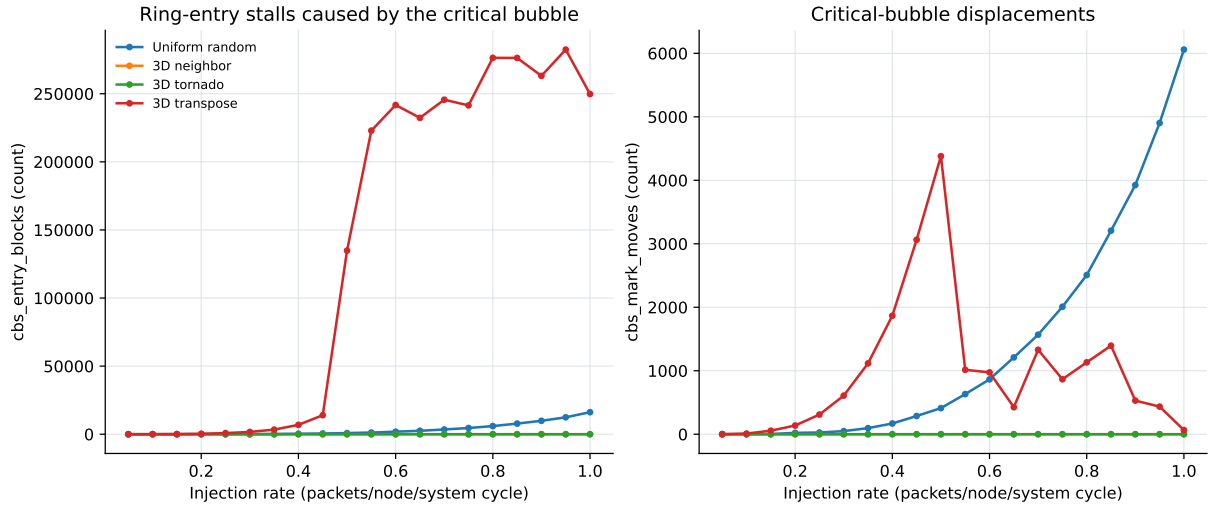


Figure 8: CBS event counters versus injection rate. Entry blocks track congestion depth; mark moves peak at the saturation knee and fall back.

A separate 200,000-cycle probe ran transpose at rate 1.0 with two and with one VC per vnet. Bare DOR triggered gem5’s 50,000-cycle deadlock detector in neither case; with one VC it was merely deeply congested, with mean latency near 77,000 cycles. This matches expectation: closing a dependency around a four-node ring under minimal routing is improbable but not impossible, so bare DOR’s observed safety is probabilistic, and CBS makes it constructive. In this deep-congestion probe, CBS’s entry throttling costs about 1.6% of injected packets (2,382,420 versus 2,420,150); no stable operating point of the 10,000-cycle main sweep shows this cost.

The comparison with the routing layer is a contrast of mechanisms, not a contest. Adaptive/escape still leads on transpose at 0.498859 because it changes the paths themselves; CBS never touches routing and only governs buffer occupancy, yet recovers part of the gap at zero VC cost and keeps DOR’s best-case tornado latency. Escape VCs partition channels; CBS maintains an occupancy invariant with all four VCs usable. The two are orthogonal answers to the same wrap-ring deadlock problem on one substrate.

## 10 Additional Validation and Limitations

The default split completed a 100,000-cycle X-opposite run at injection rate 1.0 with 0.998993 source acceptance, 0.999920 delivery ratio, 0.499456 throughput, and a 0.000989 escape-hop fraction. A vnet-2 five-flit tornado run also completed in 10,000 cycles: 31,948 packets were injected and 31,885 were received, with 478,737 adaptive hops, 110 escape hops, and 16 transitions.

The study uses synthetic traffic, one 4x4x4 size, deterministic simulator seeding, and does not include area or power modelling. The link-count analysis makes the Mesh/Torus resource difference explicit, but future work should add multiple seeds, larger dimensions, application traces, and a router/link cost model. For CBS, the scope is the single-flit control vnets, where one buffer slot equals one VC; multi-flit data vnets are neither implemented nor evaluated under CBS, and CBS is incompatible with the wormhole mode. Each directed ring carries exactly one mark, and the 200,000-cycle deadlock probe is a single run: the absence of a detector alarm for bare DOR is evidence about that run only. The result is a functioning, reproducible Garnet implementation with routing-level and flow-control-level progress arguments, not a claim of a new 3D-Torus architecture.

## 11 Reproduction

From the repository root:

```
./venv/bin/python lab/lab4/run_sweep.py
./venv/bin/python lab/lab4/plot_results.py
./venv/bin/python lab/lab4/run_vc_ablation.py --force --jobs 3
./venv/bin/python lab/lab4/plot_vc_ablation.py
./venv/bin/python lab/lab4/draw_diagrams.py
./venv/bin/python lab/lab4/topic2/run_sweep_topic2.py
./venv/bin/python lab/lab4/topic2/plot_results_topic2.py
cd lab/lab4 && pdflatex report.tex
```

The CSV files, figures, and scripts are kept in `lab/lab4/`; the CBS sweep keeps its CSV files, figures, and design notes in `lab/lab4/topic2/`; raw logs are kept under the ignored `lab/lab4/artifacts/` directory.

## References

- [1] J. Duato, “A Necessary and Sufficient Condition for Deadlock-Free Adaptive Routing in Wormhole Networks,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 6, no. 10, pp. 1055–1067, 1995.
- [2] L. Chen, R. Wang, and T. M. Pinkston, “Critical Bubble Scheme: An Efficient Implementation of Globally-Aware Network Flow Control,” in *Proc. IEEE International Parallel & Distributed Processing Symposium (IPDPS)*, 2011.